Skip to content

Chat history

You said:

die hard test list for rnadomness

ChatGPT said:

Here's the **Diehard (Marsaglia) test suite** list (commonly cited as **15 tests**) used for **randomness testing**:

1. **Birthday Spacings**
2. **Overlapping 5-Permutations**
3. **Binary Rank Test (31×31 matrices)**
4. **Binary Rank Test (32×32 matrices)**
5. **Binary Rank Test (6×8 matrices)**
6. **Bitstream Test**
7. **Overlapping Pairs Sparse Occupancy (OPSO)**
8. **Overlapping Quadruples Sparse Occupancy (OQSO)**
9. **DNA Test**
10. **Count-the-1's Test (on a stream of bytes)**
11. **Count-the-1's Test (on specific bytes)**
12. **Parking Lot Test**
13. **Minimum Distance Test**
14. **3D Spheres Test**
15. **Squeeze Test**

If you want, I can also list the **Dieharder** extended suite (many more tests) or tell you **what input format** (bits/32-bit ints) each Diehard test expects.

You said:

tell the easiet test first

ChatGPT said:

If you want the **easiest Diehard tests to implement first**, start with the ones that are basically **counting + simple distributions** (no big matrices / geometry / heavy memory):

Easiest → Harder (good beginner order)

1. **Count-the-1's (stream of bytes)**
    - Just count how many 1-bits appear in bytes/blocks and compare to what you expect.

2. **Count-the-1's (specific bytes)**
    - Same idea, just grouped differently.
3. **Squeeze test**
    - Repeatedly multiply uniforms until you "squeeze" past 1; compare the number of steps to the expected distribution.
4. **Birthday spacings**
    - Generate many integers, sort, take spacings, count repeats (Poisson approx).
5. **Minimum distance test**
    - Put random points in 2D, compute nearest-neighbor distances; compare to expected.
6. **Parking lot test** *(conceptually easy, implementation a bit longer)*
    - Randomly "park" non-overlapping circles; count how many fit.

Usually *not* the first ones to code

- **Binary rank tests (31×31, 32×32, 6×8)** (need bit-matrix rank over GF(2))
- **Bitstream / OPSO / OQSO / DNA** (more specialized + careful indexing)
- **3D spheres** (geometry + performance)

If you tell me what your RNG outputs (bits, bytes, 32-bit ints, or doubles), I'll suggest the best "first 3" and the exact input parsing for them.

You said:

and tell how much bytes or bits it tkes each tets

ChatGPT said:

Thought for 2m 18s

Below is the **data size each Diehard test consumes** in the *standard Marsaglia-style parameter settings*, assuming your input is a file of **32-bit unsigned integers** (so **1 integer = 32 bits = 4 bytes**). Where a test restarts the file for many bit-windows, I show both **per window** and the **full "all windows" total**.

Diehard tests: how many bits/bytes they use

1) Birthday spacings

- Uses **m = 2^9 = 512** "birthdays", repeated **500** times → **256,000 integers per p-value**
- Does **9 bit-windows** (bits 1–24, 2–25, …, 9–32)
- **Total:** 256,000 × 9 = **2,304,000 ints** = **9,216,000 bytes (~8.8 MiB)**

2) Overlapping 5-permutations (OPERM5)

- "Uses **1,000,000 integers, twice**"

- **Total:** 2,000,000 ints = **8,000,000 bytes (~7.6 MiB)**

3) Binary rank 31×31

- **40,000 matrices**, each needs **31 integers**
- **Total:** 40,000×31 = **1,240,000 ints** = **4,960,000 bytes (~4.7 MiB)**

4) Binary rank 32×32

- **40,000 matrices**, each needs **32 integers**
- **Total:** 1,280,000 ints = **5,120,000 bytes (~4.9 MiB)**

5) Binary rank 6×8

- **100,000 matrices**, each uses **6 integers** (one chosen byte from each)
- **Total:** 600,000 ints = **2,400,000 bytes (~2.3 MiB)**

---

6) Bitstream

- Needs a bitstring of length $\mathbf{2^{21} + 19}$ **bits** per run
- Repeated **20 times**
- **Total bits:** 20×(2,097,171) = **41,943,420 bits** = **5,242,927.5 bytes (~5.0 MiB)**

---

7) OPSO (Overlapping Pairs Sparse Occupancy)

- Builds $\mathbf{2^{21}}$ overlapping 2-letter words from $\mathbf{2^{21} + 1}$ **"keystrokes"**
- Each keystroke comes from **one 32-bit integer** (taking 10 chosen bits)
- **Per 10-bit window:** (2^{21}+1) ints ≈ **2,097,153 ints** ≈ **8,388,612 bytes (~8.0 MiB)**
- Restarts for **all 10-bit windows**: **32−10+1 = 23 windows**
- **All windows total:** ~23 × 8.39 MB ≈ **193 MB**

8) OQSO (Overlapping Quadruples Sparse Occupancy)

- Uses $\mathbf{2^{21}}$ 4-letter words from $\mathbf{2^{21} + 3}$ **keystrokes**
- Each letter is from **5 consecutive bits** of one 32-bit integer
- **Per 5-bit window:** (2^{21}+3) ints ≈ **2,097,155 ints** ≈ **8,388,620 bytes (~8.0 MiB)**
- All 5-bit windows: **32−5+1 = 28 windows**
- **All windows total:** ~28 × 8.39 MB ≈ **235 MB**

9) DNA

- Uses $\mathbf{2^{21}}$ overlapping 10-letter words from $\mathbf{2^{21} + 9}$ **keystrokes**
- Each "letter" is from **2 bits**

- **Per 2-bit window:** (2^{21}+9) ints ≈ **2,097,161 ints** ≈ **8,388,644 bytes (~8.0 MiB)**
- If you do all consecutive 2-bit windows: **32−2+1 = 31 windows**
- **All windows total:** ~31 × 8.39 MB ≈ **260 MB**

---

10) Count-the-1's (stream of bytes)

- Uses **256,000 overlapping 5-letter words** (letters are bytes)
- Needs **256,000 + 4 = 256,004 bytes** ≈ **0.244 MiB**

11) Count-the-1's (chosen bytes)

- Same **256,000** overlapping words, but each letter comes from a chosen byte of **one 32-bit integer**
- Needs **256,004 integers** = **1,024,016 bytes (~1.0 MiB)**

---

12) Parking lot

- **12,000 tries** per run
- Typically needs **2 uniforms per try** (x,y) ⇒ about **24,000 ints per run** ≈ **96,000 bytes**
- KS sample size is **10 runs**
- **Total:** ~240,000 ints = **960,000 bytes (~0.92 MiB)**

13) Minimum distance

- Done **100 times**, each time uses **8000 points** in 2D
- 2D point needs **2 uniforms** ⇒ **~16,000 ints per trial**
- **Total:** 100×16,000 = **1,600,000 ints** = **6,400,000 bytes (~6.1 MiB)**

14) 3D spheres

- Uses **4000 points**, repeated **20 times**
- 3D point needs **3 uniforms** ⇒ 12,000 ints per repeat
- **Total:** 20×12,000 = **240,000 ints** = **960,000 bytes (~0.92 MiB)**

15) Squeeze

- Produces **100,000** "j" values (each j uses a variable number of uniforms)
- So input size is **variable**:

\text{#ints}=\sum_{i=1}^{100000} j_i

(often ends up being **a few million 32-bit ints**, i.e., **tens of MB**, depending on the run)

If you tell me your file format (**raw uint32 binary**, or ASCII numbers, or bytes), I can convert everything into an exact "minimum file size you must generate" for your setup.

You said:

explain birthday test in easiest with example

ChatGPT said:

Sure — **Diehard "Birthday Spacings"** is basically: *generate many random "birthdays", sort them, look at the gaps between them, and count how often the same gap repeats.* Too many repeated gaps ⇒ not random.

The idea (in plain words)

- Imagine a year with **N** possible days (like 1…N).
- Pick **m** random days (these are the "birthdays").
- Sort them.
- Compute the **spacings** (differences) between consecutive sorted days.
- Now count **how many spacings are repeated** (same gap value appearing more than once).
- For a good RNG, the number of repeated spacings behaves like a **Poisson** random variable with a predictable mean.

Tiny example (easy to see)

Let's take a toy range **N = 50** and pick **m = 8** birthdays:

Random birthdays (unsorted):
[3, 11, 12, 20, 27, 29, 41, 46]

Sorted (already sorted here):
3, 11, 12, 20, 27, 29, 41, 46

Compute spacings (differences):

- 11−3 = 8
- 12−11 = 1
- 20−12 = 8
- 27−20 = 7
- 29−27 = 2
- 41−29 = 12
- 46−41 = 5

Spacings list:
[8, 1, 8, 7, 2, 12, 5]

Now look for repeats:

- spacing **8** occurs **twice** → that's a repeated spacing.

So here the "collision count" is:

- K = 1 repeated spacing value (or sometimes counted as "number of repeats", same idea)

If your RNG is biased/structured, you'll tend to see **too many repeated spacings** across many trials.

What Diehard actually does (standard settings)

Diehard uses large parameters so statistics become sharp:

- Choose **m = 512** birthdays per trial
- Range is **N = 2^24** (i.e., birthdays are 24-bit integers)
- Repeat this trial **500 times**
- For each trial, compute spacings and count repeats → sum over 500 trials to get **K**

Why Poisson?

For truly random picks, the expected number of repeated spacings is approximately:

$$\lambda \approx \frac{m^3}{4N}$$

For Diehard's values $m = 512,\ N = 2^{24}$:

- $m^3 = 512^3 = 134{,}217{,}728$
- $4N = 4 \cdot 2^{24} = 67{,}108{,}864$
  So

$$\lambda \approx \frac{134{,}217{,}728}{67{,}108{,}864} \approx 2$$

That means: **the number of repeated spacings per trial behaves like Poisson(≈2)**.

Over **500 trials**, the total behaves like Poisson(≈1000).

How you compute the p-value

Let:

- observed total repeats = K_{obs}
- expected mean = $\Lambda = 500\lambda \approx 1000$

Then:

$$p = \Pr(\text{Poisson}(\Lambda) \geq K_{\text{obs}})$$

(or you can do a 2-sided version, checking if it's *too small or too large*).

If **p is extremely small** (like < 1e-6) or extremely close to 1, that's suspicious.

Implementation steps (exactly what to code)

For each trial (repeat 500 times):

1. Generate m=512 random 24-bit integers in [0, 2^24).
2. Sort them.
3. Compute sp[i] = x[i+1] - x[i] for i=0..m-2.
4. Sort the spacings.
5. Count duplicates in the sorted spacings:
    - if sp[i] == sp[i-1], increment repeat counter.
6. Add repeats to total K.

That's it.

If you want, I can give you a **clean C function** that:

- reads uint32 from your .dat file,
- extracts 24-bit values (or does the "sliding window" version Diehard uses),
- outputs K, Λ, and the p-value.

You said:

not get it please

ChatGPT said:

No worries — I'll explain it like you're doing it with pen + paper.

What the Birthday Spacings test is checking

It checks **whether your random numbers create "too many repeated gaps"** after sorting.

Random numbers should look "messy".
If they have patterns, you start seeing the **same gap size** again and again more than expected.

Step-by-step with a super small example

Step 1: pick random "birthdays"

Pretend a "year" has only **20 days** (0 to 19).
Pick **8 random birthdays**:

[2, 15, 7, 8, 11, 4, 18, 6]

Step 2: sort them

Sorted:

[2, 4, 6, 7, 8, 11, 15, 18]

Step 3: compute the gaps (spacings)

Subtract neighbors:

- 4−2 = 2
- 6−4 = 2
- 7−6 = 1
- 8−7 = 1
- 11−8 = 3
- 15−11 = 4
- 18−15 = 3

So spacings are:

[2, 2, 1, 1, 3, 4, 3]

Step 4: count repeated spacings

Look for duplicates:

- spacing **2** appears twice → repeat
- spacing **1** appears twice → repeat
- spacing **3** appears twice → repeat

So repeats = **3** (there are 3 duplicated gap values)

✅ That's the whole "birthday spacings" idea.

Why is this a randomness test?

If numbers are truly random, after sorting, the gaps should vary naturally.
You'll get **some** repeated gaps, but not *too many*.

If your RNG has structure (like numbers falling into patterns), you get **lots of equal gaps**, which is suspicious.

How Diehard uses it (same steps, just bigger)

Diehard does exactly the same thing, but:

- takes **m = 512** random numbers

- each number is a “birthday” in a huge range (0 to $2^{24} - 1$)
- repeats this **500 times**
- adds up all repeats into one big count K
- compares K to what a good RNG should produce

If K is way too high or way too low → RNG fails.

One-line summary

Birthday Spacings = sort random numbers → take neighbor differences → count how often the same difference repeats.

If you tell me what you generate (bytes / uint32 / bits), I’ll write the exact tiny C code for this test in the simplest way (no “bit windows”, just the core test).

You said:

now i dsidint get how diehard will work on my byte file

ChatGPT said:

Got it — you have a **raw byte file** (like rng.dat) and you’re confused how Diehard reads it.

Key point

Classic Diehard expects a stream of 32-bit integers.
So your **byte file is consumed 4 bytes at a time** → that becomes **one uint32** number.

---

1) How Diehard reads your byte file

If your file is:

b0 b1 b2 b3 b4 b5 b6 b7 ...

Diehard groups it like:

- Int0 = (b0,b1,b2,b3)
- Int1 = (b4,b5,b6,b7)
- Int2 = (b8,b9,b10,b11)
- …

So:

- **4 bytes = 1 number = 32 bits**
- If you have **X bytes**, you have **X/4 uint32 numbers**

Endianness note (important):
Most PC implementations treat those 4 bytes as **little-endian**:

$$x = b0 + 256b1 + 256^2 b2 + 256^3 b3$$

(If you reverse byte order, your numbers change, and the test results can change.)

---

2) Now: how Birthday Spacings uses those numbers

Birthday Spacings needs "birthdays" in a fixed range, usually:

- **N = 2^{24}** (so each birthday is a **24-bit integer**)

But your input numbers are 32-bit. So Diehard **extracts 24 bits** from each 32-bit int.

The simplest extraction (one window)

Take the **lowest 24 bits**:

$$b = x \ \& \ (2^{24} - 1)$$

That gives a birthday $b \in [0, 2^{24} - 1]$.

What Diehard actually does (multiple windows)

It repeats the same test for different **bit positions** to check bias in different bits:

Window 1: bits 1–24
Window 2: bits 2–25
…
Window 9: bits 9–32

In code form (0-indexed shift):

birthday = (x >> shift) & 0xFFFFFF; // shift = 0..8

So the same file is read, and for each shift=0..8 it makes birthdays from different bits.

---

3) How many bytes your file must have (for Birthday test)

Diehard Birthday Spacings uses:

- **m = 512 birthdays per trial**
- **500 trials**
- So per bit-window it consumes:

$$512 \times 500 = 256{,}000 \text{ uint32 numbers}$$

That is:

- **256,000 ints × 4 bytes = 1,024,000 bytes (~1.0 MB)** per window

But it runs **9 windows** (shift 0..8), so total:

- **9 × 1,024,000 = 9,216,000 bytes (~8.8 MiB)**

✅ So for the Birthday test alone, generate at least **~9.3 MB** of random bytes.

---

4) In plain "workflow" terms

1. You generate a binary file rng.dat (raw bytes).
2. Diehard reads it as uint32 stream (4 bytes at a time).
3. For Birthday test:
    - pick 512 numbers → convert each to 24-bit birthday
    - sort → compute gaps → count repeated gaps
    - repeat 500 times
4. Repeat steps again for different bit shifts (0..8).

If you tell me **how you currently generate your byte file** (e.g., "I write bytes from C rand() / from ChaCha20 / from my own RNG"), I'll tell you the **exact safest format** (little-endian uint32 packing) and a tiny verification method to ensure Diehard is reading it correctly.

You said:

now after it how i will apply chi saure

ChatGPT said:

Thought for 12s

In Diehard "Birthday spacings", you first compute the **repeat count** K (how many spacing-collisions you saw). After that, you can apply **chi-square** in two easy ways:

Option A (easiest): Chi-square on the p-values (uniformity test)

Step 1: For each trial, convert K to a p-value

Under a good RNG, the collision count per trial is approximately

$$K \sim \text{Poisson}(\lambda), \lambda \approx \frac{m^3}{4N}.$$

In Diehard's standard birthday test: $m = 512,\ N = 2^{24} \Rightarrow \lambda \approx 2.$

For each trial i, you get K_i. Turn it into a p-value using the Poisson CDF:

- upper-tail p-value:

$$p_i = \Pr(\text{Poisson}(\lambda) \geq K_i) = 1 - F_{\text{Pois}}(K_i - 1; \lambda).$$

Now if the RNG is good, these p_i's should look **Uniform(0,1)**.

Step 2: Bin the p-values and do chi-square

Take T trials (e.g., $T = 500$). Choose B bins (common: $B = 10$):

- Bin 1: $[0,0.1)$
- Bin 2: $[0.1,0.2)$
- …
- Bin 10: $[0.9, 1.0]$

Count how many p-values fall in each bin → observed counts O_j.

Expected count in each bin:

$$E_j = \frac{T}{B}.$$

Chi-square statistic:

$$\chi^2 = \sum_{j=1}^{B} \frac{(O_j - E_j)^2}{E_j}.$$

Degrees of freedom:

$$\text{df} = B - 1.$$

Then compute the chi-square p-value:

$$p_{\chi^2} = \Pr(\chi^2_{\text{df}} \geq \chi^2_{\text{obs}}).$$

✅ If p_{χ^2} is extremely small (like $<10^{-6}$) or extremely close to 1 repeatedly, that's suspicious.

Why this is easiest: you don't worry about Poisson bins; you just check whether p-values are uniform.

Option B: Chi-square directly on the Poisson counts K

Instead of converting to p-values, you can test whether the histogram of K_i matches a Poisson(λ).

Step 1: Choose bins for K

Example bins:

- 0, 1, 2, 3, 4, and ≥ 5

(You want expected counts ≥ ~5 in each bin; merge the tail if needed.)

Step 2: Compute expected counts from Poisson probabilities

For bin k:

$$E_k = T \cdot \Pr(K = k) = T \cdot e^{-\lambda} \frac{\lambda^k}{k!}.$$

Tail bin:

$$E_{\geq 5} = T \cdot \Pr(K \geq 5) = T \cdot \left(1 - \sum_{k=0}^{4} e^{-\lambda} \frac{\lambda^k}{k!}\right).$$

Step 3: Chi-square statistic

$$\chi^2 = \sum_{\text{bins}} \frac{(O-E)^2}{E}.$$

Degrees of freedom:

- If λ is **fixed** (like $\lambda = 2$ from theory):
$$\text{df} = \#\text{bins} - 1$$
- If you **estimate** λ from data (using $\hat{\lambda} = \bar{K}$):
$$\text{df} = \#\text{bins} - 1 - 1 = \#\text{bins} - 2$$

Which should you use?

- If you're implementing it yourself: **Option A (p-values → chi-square uniformity)** is simplest and robust.
- If you want to stay closest to the "Poisson collision" story: **Option B** is fine.

If you tell me what you are doing exactly:

- are you getting **one** K **per trial** (500 values), or
- one K per **bit-window** (9 sets of 500),
I'll write the exact loop (and the exact λ) you should use for your setup.

You said:

give code in c

ChatGPT said:

Thought for 29s

```
/* Diehard-style Birthday Spacings test (core) + Chi-square on p-values Input: raw binary byte file (rng.dat) read as little-endian uint32 words. For each shift in [0..8], we: - run TRIALS trials - each trial uses M uint32 words -> birthdays are 24-bit: (x >> shift) & 0xFFFFFF - compute circular spacings (M spacings), count repeated spacings => K - convert K to p-value using Poisson(lambda) - apply chi-square uniformity test on p-values (10 bins) Build: gcc -O2 -std=c11 birthday_chi.c -lm Run: ./a.out rng.dat */ #include <stdio.h> #include <stdlib.h> #include <stdint.h> #include <math.h> #include <string.h> #ifndef M_PI #define M_PI 3.14159265358979323846 #endif /* ---------- PARAMETERS (edit if you want) ---------- */ #define M 512 /* birthdays per trial */ #define TRIALS 500 /* number of trials */ #define NBINS 10 /* bins for chi-square on p-values */ #define N24 (1u << 24) /* range size N = 2^24 */ #define MASK24 0xFFFFFFu /* ---------- little-endian uint32 read ---------- */ static int read_u32_le(FILE *fp, uint32_t *out) { unsigned char b[4]; if (fread(b, 1, 4, fp) != 4) return 0; *out = (uint32_t)b[0] | ((uint32_t)b[1] << 8) | ((uint32_t)b[2] << 16) | ((uint32_t)b[3] << 24); return 1; } /* ---------- qsort comparators ---------- */ static int cmp_u32(const void *a, const void *b) { uint32_t x = *(const uint32_t*)a, y = *(const uint32_t*)b; return (x > y) - (x < y); } /* ---------- Poisson survival: P(X >= k) ---------- We compute CDF up to k-1 by recurrence: p0 = exp(-lambda) p_{i+1} = p_i * lambda/(i+1) */ static double poisson_sf(int k, double lambda) { if (k <= 0) return 1.0; if (lambda <= 0.0) return 0.0; double p = exp(-lambda); /* P(X=0) */ double cdf = p; /* sum_{i=0}^0 p_i */ for (int i = 0; i < k-1; i++) { /* move from p_i to p_{i+1} */ p *= lambda / (double)(i + 1 + 0); /* careful with indexing below */ /* The line above is wrong indexing if left like this; do it cleanly: */ } /* We'll redo properly below (keep function correct) */ return 0.0; } /* Correct Poisson SF implementation */ static double poisson_sf_correct(int k, double lambda) { if (k <= 0) return 1.0; double p = exp(-lambda); /* p0 */ double cdf = p; for (int i = 0; i < k-1; i++) { /* p_{i+1} from p_i */ p *= lambda / (double)(i + 1); cdf += p; /* tiny guard: if cdf is essentially 1, sf ~ 0 */ if (cdf >= 1.0 - 1e-15) return 0.0; } double sf = 1.0 - cdf; if (sf < 0.0) sf = 0.0; return sf; } /* ---------- Chi-square p-value via regularized upper gamma Q(a,x) ---------- We need p = P(ChiSq_df >= chi2) = Q(df/2, chi2/2). Implementation adapted from classic Numerical Recipes style (series + continued fraction). */ static double gammln(double xx) { static const double cof[6] = { 76.18009172947146, -86.50532032941677, 24.01409824083091, -1.231739572450155, 0.1208650973866179e-2,-0.5395239384953e-5 }; double x = xx, y = xx; double tmp = x + 5.5; tmp -= (x + 0.5) * log(tmp); double ser = 1.000000000190015; for (int j = 0; j < 6; j++) ser += cof[j] / ++y; return -tmp + log(2.5066282746310005 * ser / x); } static void gser(double *gamser, double a, double x, double *gln) { const int ITMAX = 200; const double EPS = 3.0e-14; *gln = gammln(a); if (x <= 0.0) { *gamser = 0.0; return; } double sum = 1.0 / a; double del = sum; double ap = a; for (int n = 1; n <= ITMAX; n++) { ap += 1.0; del *= x / ap; sum += del; if (fabs(del) < fabs(sum) * EPS) break; } *gamser = sum * exp(-x + a * log(x) - (*gln)); } static void gcf(double *gammcf, double a, double x, double *gln) { const int ITMAX = 200; const double EPS = 3.0e-14; const double FPMIN = 1.0e-300; *gln = gammln(a); double b = x + 1.0 - a; double c = 1.0 / FPMIN; double d = 1.0 / b; double h = d; for (int i = 1; i <= ITMAX; i++) { double an = -i * (i - a); b += 2.0; d = an * d + b; if (fabs(d) < FPMIN) d = FPMIN; c = b + an / c; if (fabs(c) < FPMIN) c = FPMIN; d = 1.0 / d; double del = d * c; h *= del; if (fabs(del - 1.0) < EPS) break; } *gammcf = exp(-x + a * log(x) - (*gln)) * h; } /* Regularized upper incomplete gamma Q(a,x) */ static double gammq(double a, double x) { if (x < 0.0 || a <= 0.0) return 0.0; double gamser, gammcf, gln; if (x < a + 1.0) { gser(&gamser, a, x, &gln); return 1.0 - gamser; /* Q = 1 - P */ } else { gcf(&gammcf, a, x, &gln); return gammcf; /* Q directly */ } } /* ---------- Birthday spacings: one shift ---------- */ static int birthday_one_shift(FILE *fp, int shift, double lambda, int bins[NBINS], double *meanK_out) { uint32_t birthdays[M]; uint32_t spacings[M]; memset(bins, 0, NBINS * sizeof(int)); long long sumK = 0; for (int t = 0; t < TRIALS; t++) { /* read M uint32 -> birthdays */ for (int i = 0; i < M; i++) { uint32_t x; if (!read_u32_le(fp, &x)) { fprintf(stderr, "EOF: need more data (shift=%d, trial=%d, i=%d)\n", shift, t, i); return 0; } birthdays[i] = (x >> shift) & MASK24; } /* sort
```

birthdays */ qsort(birthdays, M, sizeof(uint32_t), cmp_u32); /* circular spacings (M of them) */ for (int i = 0; i < M - 1; i++) spacings[i] = birthdays[i + 1] - birthdays[i]; spacings[M - 1] = (birthdays[0] + N24) - birthdays[M - 1]; /* sort spacings */ qsort(spacings, M, sizeof(uint32_t), cmp_u32); /* count repeated spacings */ int K = 0; for (int i = 1; i < M; i++) { if (spacings[i] == spacings[i - 1]) K++; } sumK += K; /* Poisson p-value: upper tail P(X >= K) */ double p = poisson_sf_correct(K, lambda); /* bin p into 10 equal bins */ int b = (int)floor(p * NBINS); if (b < 0) b = 0; if (b >= NBINS) b = NBINS - 1; bins[b]++; } *meanK_out = (double)sumK / (double)TRIALS; return 1; } static void chi_square_uniform(const int bins[NBINS], int n, double *chi2_out, double *p_out) { double E = (double)n / (double)NBINS; double chi2 = 0.0; for (int i = 0; i < NBINS; i++) { double O = (double)bins[i]; double d = O - E; chi2 += (d * d) / E; } int df = NBINS - 1; double p = gammq(0.5 * df, 0.5 * chi2); /* upper-tail */ *chi2_out = chi2; *p_out = p; } int main(int argc, char **argv) { if (argc < 2) { fprintf(stderr, "Usage: %s rng.dat\n", argv[0]); return 1; } FILE *fp = fopen(argv[1], "rb"); if (!fp) { perror("fopen"); return 1; } /* Diehard lambda approximation: lambda ≈ m^3/(4N), with N=2^24 */ double lambda = ((double)M * (double)M * (double)M) / (4.0 * (double)N24); printf("Birthday Spacings test (M=%d, TRIALS=%d, N=2^24), lambda≈%.6f\n", M, TRIALS, lambda); printf("Input is read as little-endian uint32 from the byte file.\n\n"); for (int shift = 0; shift <= 8; shift++) { rewind(fp); int bins[NBINS]; double meanK = 0.0; if (!birthday_one_shift(fp, shift, lambda, bins, &meanK)) { fclose(fp); return 2; } double chi2, pchi; chi_square_uniform(bins, TRIALS, &chi2, &pchi); printf("Shift=%d (using bits %d..%d for 24-bit birthdays):\n", shift, shift+1, shift+24); printf(" mean(K)=%.4f\n", meanK); printf(" p-value bins (10 equal bins): "); for (int i = 0; i < NBINS; i++) printf("%d%s", bins[i], (i==NBINS-1) ? "" : ","); printf("\n"); printf(" ChiSq=%.4f, df=%d, ChiSq-p=%.6g\n\n", chi2, NBINS-1, pchi); } fclose(fp); return 0; }

What you do with it

1. Generate a **raw binary** file rng.dat (just bytes).
2. Compile & run:

gcc -O2 -std=c11 birthday_chi.c -lm ./a.out rng.dat

How big your file should be (minimum)

For **one shift** it needs M*TRIALS = 512*500 = 256000 uint32 numbers ⇒ 256000*4 = 1,024,000 bytes (~1 MB).
Since the code **rewinds** and reuses the same data for shifts 0..8, **~1 MB is enough** to run all shifts here.

If you want the “more strict” approach (different fresh data for each shift), tell me — I’ll modify the code to *not rewind* and then you’ll need ~9 MB.

You said:

#include "header.h" static real chi_fit; static int dgf; int ucmpr(const void *u1, const void *u2); /*compute the statistic of goodness of fit to a Poisson distribution*/ real P_fit(real lambda, counter *obs, int no_obs) { counter *f, dim=no_obs/5; register int i=-1; register counter j, k=0; real rest=no_obs, *Ef; f=(counter*)calloc(dim, sizeof(counter)); Ef=(real*)calloc(dim, sizeof(real)); qsort(obs, no_obs, sizeof(counter), ucmpr); for(j=0; j<dim; ++j){ while(Ef[j]<5){ ++i; Ef[j]+=no_obs*Poisson(lambda, i); } while(obs[k]<=i){ ++f[j]; ++k; } rest-=Ef[j]; if(rest<5){ Ef[j]+=rest; f[j]+=no_obs-k; break; } } chi_fit=0; for(i=0; i<=j; ++i){ chi_fit+=(f[i]-Ef[i])*(f[i]-Ef[i])/Ef[i]; } dgf=j; /* cfree(f, dim, sizeof(counter)); cfree(Ef, dim, sizeof(real)); */ return 1-Chisq(dgf,chi_fit); } void bday(char *filename) { const counter no_obs=500, no_bday=1024, no_bits=24; register const

counter mask=pow(2,no_bits)-1; const real lambda=pow(no_bday,3)/(4*pow(2,no_bits)); register int rt; register int i, k, sum; uniform *bdspace; counter no_dup, *obs; real pvalue, *p; puts("\n\t|----------
----------|"); puts("\t| This is the BIRTHDAY SPACINGS TEST |"); puts("\t|Choose m birthdays in a \"year\" of n days. List the spacings |"); puts("\t|between the birthdays. Let j be the number of values that |"); puts("\t|occur more than once in that list, then j is asymptotically |"); puts("\t|Poisson distributed with mean m^3/(4n). Experience shows n |"); puts("\t|must be quite large, say n>=2^18, for comparing the results |"); puts("\t|to the Poisson distribution with that mean. This test uses |"); puts("\t|n=2^24 and m=2^10, so that the underlying distribution for j |"); puts("\t|is taken to be Poisson with lambda=2^30/(2^26)=16. A sample |"); puts("\t|of 200 j''s is taken, and a chi-square goodness of fit test |"); puts("\t|provides a p value. The first test uses bits 1-24 (counting |"); puts("\t|from the left) from integers in the specified file. Then the|"); puts("\t|file is closed and reopened, then bits 2-25 of the same inte-|"); puts("\t|gers are used to provide birthdays, and so on to bits 9-32. |"); puts("\t|Each set of bits provides a p-value, and the nine p-values |"); puts("\t|provide a sample for a KSTEST. |"); puts("\t|----------
---------- |\n"); printf("\t\tRESULTS OF BIRTHDAY SPACINGS TEST FOR %s\n", filename); printf("\t(no_bdays=%lu, no_days/yr=2^%lu,",no_bday, no_bits); printf(" lambda=%.2f, sample size=%lu)\n\n", lambda, no_obs); printf("\tBits used\tmean\t\tchisqr\t\tp-value\n"); obs=(counter*)malloc(no_obs*sizeof(counter)); p=(real*)malloc((32-no_bits+1)*sizeof(real)); bdspace=(uniform*)malloc(no_bday*sizeof(uniform)); #define GETDAY ((uni(filename) >> rt) & mask) for(rt=32-no_bits; rt>=0; --rt){ sum=0; for(k=0; k<no_obs; ++k){ for(i=0; i<no_bday; ++i){ bdspace[i]=GETDAY; } qsort(bdspace, no_bday, sizeof(uniform), ucmpr); for(i=no_bday-1; i>=1; --i){ bdspace[i]-=bdspace[i-1]; } qsort(bdspace, no_bday, sizeof(uniform), ucmpr); no_dup=0; for(i=1; i<no_bday; ++i){ if(bdspace[i]!=bdspace[i-1]) continue; else ++no_dup; } sum+=no_dup; obs[k]=no_dup; } uni("close"); p[rt]=P_fit(lambda, obs, no_obs); printf("\t %lu to %d", 33-no_bits-rt,32-rt); printf("\t%.2f\t\t%.4f\t\t%f\n", (real)sum/no_obs, chi_fit, p[rt]); } /*clean up*/ free(bdspace); free(obs); pvalue=KStest(p,32-no_bits+1); printf("\n\t\t\tdegree of freedoms is: %d\n", dgf); puts("\t----------
----------"); printf("\t\tp-value for KStest on those %lu p-values: %f",32-no_bits+1,pvalue); puts("\n"); free(p); return; } /*main() { char ch; bday("binc"); return; }*/ is the code is same u give

ChatGPT said:

Thought for 17s

It's the **same *Diehard Birthday Spacings* idea**, but it is **not the same as the C code I gave you**. Your code is basically the **original Diehard-style implementation** (Poisson GOF + KS over 9 bit-windows). Mine was a **simplified core test** + **chi-square on p-values**.

What is the same (conceptually)

- Read random data as **32-bit integers** (uni(filename) in your code; my code reads 4 bytes → uint32_t).
- Form 24-bit "birthdays" using **bit windows**:

$$\text{day} = (x \gg rt)\ \&\ 0xFFFFFF$$

- For each window, repeat many trials, compute "duplicate spacings", turn it into a p-value.

Key differences

1) m (number of birthdays per trial)

- **Your code:** no_bday = 1024 (= 2^{10})
- **My code:** M = 512

So the test strength and lambda change.

2) λ (Poisson mean)

- **Your code:**

$$\lambda = \frac{1024^3}{4 \cdot 2^{24}} = 16$$

- **My code:**

$$\lambda = \frac{512^3}{4 \cdot 2^{24}} \approx 2$$

So even if everything else matched, the expected collision counts differ a lot.

3) How chi-square is applied

- **Your code:** does a **chi-square goodness-of-fit** test of the *distribution of* $\{K_i\}$ to **Poisson(λ)** via P_fit().
    - It bins Poisson probabilities until expected count ≥ 5, then computes:

$$\chi^2 = \sum \frac{(O-E)^2}{E}$$

- **My code:** I did **chi-square uniformity** on the **p-values** (10 equal bins on [0,1]) — simpler, not the original Diehard method.

4) KS test stage

- **Your code:** After getting **9 p-values** (for bit windows), it runs **KStest(p, 9)**.
- **My code:** I did **no KS across shifts** (I only printed chi-square p per shift).

5) Data reuse across windows

- **Your code:** closes/reopens the RNG (uni("close")) each window (i.e., re-reads the same file again per window).
- **My code:** used rewind(fp) (same effect).

Important possible issue in your posted code (check this!)

In birthday spacings, you normally need the **wrap-around spacing** too:

$$s_{m-1} = (x_0 + N) - x_{m-1}$$

In your loop:

for(i=no_bday-1; i>=1; --i){ bdspace[i]-=bdspace[i-1]; }

bdspace[0] is left as the **smallest birthday**, not the wrap-around spacing.
Unless uniform/uni()/your original library does something special elsewhere, this looks like a **missing circular spacing** and can change the statistics.

✅ In my code I explicitly did:

spacings[M-1] = (birthdays[0] + N24) - birthdays[M-1];

Bottom line

- **Same test idea:** yes.
- **Same code as I gave: no** (different m, different lambda, different chi-square method, and your code includes KS step).
- Also **verify wrap-around spacing**, because that's a common (and serious) bug.

If you want, paste your uni() and uniform/counter typedefs (or header.h parts), and I'll tell you **exactly** whether your spacing computation is correct for your implementation.

You said:

so u give coorect code in c everything is clear and i have file outputdat

ChatGPT said:

Thought for 18s

Below is a **correct, self-contained C program** that implements the **full Diehard "Birthday Spacings" test** the standard way:

- Reads your **raw byte file** (output.dat) as **little-endian uint32**.
- Uses **n = 2^24**, **m = 2^10 = 1024 birthdays per sample**, **500 samples**.
- For each **24-bit window** (bits 1–24, 2–25, …, 9–32 ⇒ 9 windows):
    - generates 500 values of **K = (# duplicated spacings)** (with **wrap-around spacing included** ✅)
    - computes a **chi-square goodness-of-fit** to **Poisson(λ=16)**
    - prints **mean K**, **chi-square**, **p-value**
- Then performs a **KS test** on the 9 p-values and prints the KS p-value.

Minimum file size needed (because we rewind and reuse the same data for each window, like Diehard):

1024×500 uint32 = 512,000 uint32 = **2,048,000 bytes (~2 MB)**.
If your file is bigger, that's fine.

/* Diehard Birthday Spacings Test (full style): - n = 2^24 days - m = 1024 birthdays per sample - 500 samples per bit-window - 9 windows: bits 1-24 ... 9-32 (i.e., shifts 8..0 from MSB view) - For each window: chi-square GOF of K to Poisson(lambda=16) - KS test on the 9 resulting p-values Build: gcc -O2 -std=c11 diehard_bday.c -lm Run: ./a.out output.dat */ #include <stdio.h> #include <stdlib.h> #include <stdint.h> #include <math.h> #include <string.h> /* ---------- Parameters (Diehard standard for this test) ---------- */ #define NO_OBS 500 #define NO_BDAY 1024 #define NO_BITS 24 #define N_DAYS (1u << 24) #define MASK24 0xFFFFFFu /* ---------- Little-endian uint32 reader from raw byte file ---------- */ static int read_u32_le(FILE *fp, uint32_t *out) { unsigned char b[4]; if (fread(b, 1, 4, fp) != 4) return 0; *out = (uint32_t)b[0] | ((uint32_t)b[1] << 8) | ((uint32_t)b[2] << 16) | ((uint32_t)b[3] << 24); return 1; } /* ---------- qsort comparator ---------- */ static int cmp_u32(const void *a, const void *b) { uint32_t x = *(const uint32_t*)a; uint32_t y = *(const uint32_t*)b; return (x > y) - (x < y); } /* ---------- log-gamma helpers for chi-square p-value ---------- We need upper-tail p = P(ChiSq_df >= x) = Q(df/2, x/2). Implemented via regularized upper incomplete gamma Q(a,x). */ static double gammln(double xx) { static const double cof[6] = { 76.18009172947146, -86.50532032941677, 24.01409824083091, -1.231739572450155, 0.1208650973866179e-2,-0.5395239384953e-5 }; double x = xx, y = xx; double tmp = x + 5.5; tmp -= (x + 0.5) * log(tmp); double ser = 1.000000000190015; for (int j = 0; j < 6; j++) ser += cof[j] / ++y; return -tmp + log(2.5066282746310005 * ser / x); } static void gser(double *gamser, double a, double x, double *gln) { const int ITMAX = 200; const double EPS = 3.0e-14; *gln = gammln(a); if (x <= 0.0) { *gamser = 0.0; return; } double sum = 1.0 / a; double del = sum; double ap = a; for (int n = 1; n <= ITMAX; n++) { ap += 1.0; del *= x / ap; sum += del; if (fabs(del) < fabs(sum) * EPS) break; } *gamser = sum * exp(-x + a * log(x) - (*gln)); } static void gcf(double *gammcf, double a, double x, double *gln) { const int ITMAX = 200; const double EPS = 3.0e-14; const double FPMIN = 1.0e-300; *gln = gammln(a); double b = x + 1.0 - a; double c = 1.0 / FPMIN; double d = 1.0 / b; double h = d; for (int i = 1; i <= ITMAX; i++) { double an = -i * (i - a); b += 2.0; d = an * d + b; if (fabs(d) < FPMIN) d = FPMIN; c = b + an / c; if (fabs(c) < FPMIN) c = FPMIN; d = 1.0 / d; double del = d * c; h *= del; if (fabs(del - 1.0) < EPS) break; } *gammcf = exp(-x + a * log(x) - (*gln)) * h; } static double gammq(double a, double x) { if (x < 0.0 || a <= 0.0) return 0.0; double gamser, gammcf, gln; if (x < a + 1.0) { gser(&gamser, a, x, &gln); return 1.0 - gamser; } else { gcf(&gammcf, a, x, &gln); return gammcf; } } /* Upper-tail chi-square p-value */ static double chisq_sf(int df, double chi2) { return gammq(0.5 * df, 0.5 * chi2); } /* ---------- Poisson pmf via recurrence (stable for small/medium k) ---------- */ static double poisson_pmf(int k, double lambda) { /* compute e^-lambda * lambda^k / k! using recurrence from 0 */ double p0 = exp(-lambda); if (k == 0) return p0; double p = p0; for (int i = 1; i <= k; i++) { p *= lambda / (double)i; } return p; } /* ---------- Chi-square GOF of observed K's to Poisson(lambda) ---------- We follow the Diehard-style rule: combine categories so each expected >= 5. Inputs: obs[NO_OBS] = K values (duplicates per sample) Outputs: chi2, df, p-value. */ static void poisson_chisq_gof(const int *obs, int n, double lambda, double *chi2_out, int *df_out, double *p_out) { /* Make a sorted copy of obs */ int *x = (int*)malloc((size_t)n * sizeof(int)); if (!x) { fprintf(stderr, "malloc failed\n"); exit(1); } memcpy(x, obs, (size_t)n * sizeof(int)); /* sort */ /* simple qsort comparator for int */ int cmp_int(const void *a, const void *b) { int A = *(const int*)a, B = *(const int*)b; return (A > B) - (A < B); } qsort(x, (size_t)n, sizeof(int), cmp_int); /* Build bins by increasing k, accumulating expected until >=5 */ double chi2 = 0.0; int bins = 0; int idx = 0; /* how many obs consumed */ int k = -1; /* current Poisson k */ double exp_in_bin = 0.0; int obs_in_bin = 0; while (idx < n) { /* extend bin until expected >= 5 */ while (exp_in_bin < 5.0) { k++; exp_in_bin +=

```
(double)n * poisson_pmf(k, lambda); /* add all observations equal to k */ while (idx < n && x[idx] <= k) { obs_in_bin++; idx++; } /* if we are near the end, dump the tail into this bin */ double remaining_expected = (double)n - (double)(bins == 0 ? 0 : 0); /* not used */ (void)remaining_expected; if (idx >= n) break; } /* If remaining expected mass is < 5, merge rest into this bin: easiest way: if remaining observations are few, keep merging. */ /* We handle this by checking if remaining expected (tail) would be <5: approximate tail expected = n*(1 - sum_{i=0..k} pmf(i)). We'll compute tail cdf roughly by summing pmf up to k. */ /* For simplicity & correctness: if observations remain but we are close, we'll continue making bins normally; expected will not stay tiny with lambda=16, n=500. */ /* finalize this bin */ if (exp_in_bin > 0.0) { double diff = (double)obs_in_bin - exp_in_bin; chi2 += (diff * diff) / exp_in_bin; bins++; } /* reset bin counters */ exp_in_bin = 0.0; obs_in_bin = 0; } /* degrees of freedom = bins - 1 (lambda fixed) */ int df = bins - 1; if (df < 1) df = 1; double p = chisq_sf(df, chi2); free(x); *chi2_out = chi2; *df_out = df; *p_out = p; } /* ---------- One-sample KS test for Uniform(0,1) ---------- */ static double ks_pvalue_uniform(const double *pvals, int n) { double *x = (double*)malloc((size_t)n * sizeof(double)); if (!x) { fprintf(stderr, "malloc failed\n"); exit(1); } memcpy(x, pvals, (size_t)n * sizeof(double)); int cmp_double(const void *a, const void *b) { double A = *(const double*)a, B = *(const double*)b; return (A > B) - (A < B); } qsort(x, (size_t)n, sizeof(double), cmp_double); double D = 0.0; for (int i = 0; i < n; i++) { double xi = x[i]; double cdf_emp_hi = (double)(i + 1) / (double)n; double cdf_emp_lo = (double)(i) / (double)n; double d1 = fabs(cdf_emp_hi - xi); double d2 = fabs(xi - cdf_emp_lo); if (d1 > D) D = d1; if (d2 > D) D = d2; } free(x); /* Kolmogorov asymptotic p-value: p ≈ 2 Σ (-1)^{k-1} exp(-2 k^2 D^2 n) */ double nn = (double)n; double z = D * sqrt(nn); /* Use the exact series in terms of D and n */ double p = 0.0; for (int k = 1; k <= 200; k++) { double term = exp(-2.0 * (double)k * (double)k * D * D * nn); if (k % 2 == 1) p += 2.0 * term; else p -= 2.0 * term; if (term < 1e-12) break; } if (p < 0.0) p = 0.0; if (p > 1.0) p = 1.0; (void)z; return p; } /* ---------- Main Birthday Spacings per bit-window ---------- */ static void run_bday_window(FILE *fp, int shift, double lambda, double *meanK_out, double *chi2_out, int *df_out, double *p_out) { uint32_t bday[NO_BDAY]; uint32_t space[NO_BDAY]; int obs[NO_OBS]; long long sumK = 0; for (int t = 0; t < NO_OBS; t++) { for (int i = 0; i < NO_BDAY; i++) { uint32_t x; if (!read_u32_le(fp, &x)) { fprintf(stderr, "EOF: not enough data for shift=%d (trial=%d, i=%d)\n", shift, t, i); exit(2); } bday[i] = (x >> shift) & MASK24; } qsort(bday, NO_BDAY, sizeof(uint32_t), cmp_u32); /* Compute spacings INCLUDING wrap-around spacing (critical) */ for (int i = 0; i < NO_BDAY - 1; i++) { space[i] = bday[i + 1] - bday[i]; } space[NO_BDAY - 1] = (bday[0] + N_DAYS) - bday[NO_BDAY - 1]; qsort(space, NO_BDAY, sizeof(uint32_t), cmp_u32); /* Count duplicates (total repeats) */ int K = 0; for (int i = 1; i < NO_BDAY; i++) { if (space[i] == space[i - 1]) K++; } obs[t] = K; sumK += K; } *meanK_out = (double)sumK / (double)NO_OBS; /* Chi-square goodness-of-fit of obs[] to Poisson(lambda) */ poisson_chisq_gof(obs, NO_OBS, lambda, chi2_out, df_out, p_out); } int main(int argc, char **argv) { if (argc < 2) { fprintf(stderr, "Usage: %s output.dat\n", argv[0]); return 1; } const char *fname = argv[1]; FILE *fp = fopen(fname, "rb"); if (!fp) { perror("fopen"); return 1; } /* lambda = m^3/(4n) with m=1024, n=2^24 => 16 */ double lambda = (double)NO_BDAY * (double)NO_BDAY * (double)NO_BDAY / (4.0 * (double)N_DAYS); printf("\nDiehard Birthday Spacings Test on file: %s\n", fname); printf("n = 2^%d = %u, m = %d, samples = %d, lambda = %.6f\n\n", NO_BITS, N_DAYS, NO_BDAY, NO_OBS, lambda); printf("Bits used (from MSB view)\tmean(K)\t\tchisq\t\tdf\tp-value\n"); double pvals[9]; int idx = 0; /* Diehard uses bits 1-24, 2-25, ... 9-32. With uint32 x, that corresponds to shifts 8 down to 0 if you imagine MSB-indexing. But mathematically it is just 9 consecutive 24-bit windows. We implement shift = 0..8 (LSB upward). This is standard in many implementations. If you want "from the left" exactly, reverse the shift order—KS result is the same set. */ for (int shift = 0; shift <= 8; shift++) { rewind(fp); double meanK, chi2, p; int df; run_bday_window(fp, shift, lambda, &meanK, &chi2, &df, &p); /* report bits (1-based human
```

display) */ int bit_lo = shift + 1; int bit_hi = shift + NO_BITS; printf("%2d to %2d\t\t\t\t%.3f\t\t%.4f\t\t%d\t%.6g\n", bit_lo, bit_hi, meanK, chi2, df, p); pvals[idx++] = p; } fclose(fp); /* KS test on the 9 p-values */ double ks_p = ks_pvalue_uniform(pvals, 9); printf("\nKS test on the 9 p-values: KS p-value = %.6g\n\n", ks_p); return 0; }

How to run

gcc -O2 -std=c11 diehard_bday.c -lm ./a.out outputdat

Notes (important)

- Your file **must be raw bytes** (not ASCII numbers).
- This code assumes **little-endian** packing for uint32 (most common).
- The **wrap-around spacing** is included (this is where many "almost-diehard" codes go wrong).

If you want, paste **how you generated** outputdat (did you write bytes directly, or printed integers as text?), and I'll tell you immediately whether you should read as **binary** or **ASCII** and adjust the reader accordingly.


```
/*
  Diehard Birthday Spacings Test (full style):
  - n = 2^24 days
  - m = 1024 birthdays per sample
  - 500 samples per bit-window
  - 9 windows: bits 1-24 ... 9-32 (i.e., shifts 8..0 from MSB view)
  - For each window: chi-square GOF of K to Poisson(lambda=16)
  - KS test on the 9 resulting p-values

  Build:  gcc -O2 -std=c11 diehard_bday.c -lm
  Run:    ./a.out output.dat
*/

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <math.h>
#include <string.h>
```

```
/* ---------------- Parameters (Diehard standard for this test) ---------------- */
#define NO_OBS   500
#define NO_BDAY  1024
#define NO_BITS  24
#define N_DAYS   (1u << 24)
#define MASK24   0xFFFFFFu

/* ---------------- Little-endian uint32 reader from raw byte file ---------------- */
static int read_u32_le(FILE *fp, uint32_t *out) {
    unsigned char b[4];
    if (fread(b, 1, 4, fp) != 4) return 0;
    *out = (uint32_t)b[0]
         | ((uint32_t)b[1] << 8)
         | ((uint32_t)b[2] << 16)
         | ((uint32_t)b[3] << 24);
    return 1;
}

/* ---------------- qsort comparator ---------------- */
static int cmp_u32(const void *a, const void *b) {
    uint32_t x = *(const uint32_t*)a;
    uint32_t y = *(const uint32_t*)b;
    return (x > y) - (x < y);
}

/* ---------------- log-gamma helpers for chi-square p-value ----------------
   We need upper-tail p = P(ChiSq_df >= x) = Q(df/2, x/2).
   Implemented via regularized upper incomplete gamma Q(a,x).
*/
static double gammln(double xx) {
    static const double cof[6] = {
```

```
        76.18009172947146,    -86.50532032941677,
        24.01409824083091,    -1.231739572450155,
        0.1208650973866179e-2,-0.5395239384953e-5
    };
    double x = xx, y = xx;
    double tmp = x + 5.5;
    tmp -= (x + 0.5) * log(tmp);
    double ser = 1.000000000190015;
    for (int j = 0; j < 6; j++) ser += cof[j] / ++y;
    return -tmp + log(2.5066282746310005 * ser / x);
}

static void gser(double *gamser, double a, double x, double *gln) {
    const int ITMAX = 200;
    const double EPS = 3.0e-14;
    *gln = gammln(a);
    if (x <= 0.0) { *gamser = 0.0; return; }

    double sum = 1.0 / a;
    double del = sum;
    double ap = a;

    for (int n = 1; n <= ITMAX; n++) {
        ap += 1.0;
        del *= x / ap;
        sum += del;
        if (fabs(del) < fabs(sum) * EPS) break;
    }
    *gamser = sum * exp(-x + a * log(x) - (*gln));
}
```

```
static void gcf(double *gammcf, double a, double x, double *gln) {
    const int ITMAX = 200;
    const double EPS = 3.0e-14;
    const double FPMIN = 1.0e-300;
    *gln = gammln(a);

    double b = x + 1.0 - a;
    double c = 1.0 / FPMIN;
    double d = 1.0 / b;
    double h = d;

    for (int i = 1; i <= ITMAX; i++) {
        double an = -i * (i - a);
        b += 2.0;
        d = an * d + b;
        if (fabs(d) < FPMIN) d = FPMIN;
        c = b + an / c;
        if (fabs(c) < FPMIN) c = FPMIN;
        d = 1.0 / d;
        double del = d * c;
        h *= del;
        if (fabs(del - 1.0) < EPS) break;
    }
    *gammcf = exp(-x + a * log(x) - (*gln)) * h;
}

static double gammq(double a, double x) {
    if (x < 0.0 || a <= 0.0) return 0.0;
    double gamser, gammcf, gln;
    if (x < a + 1.0) {
        gser(&gamser, a, x, &gln);
```

```
        return 1.0 - gamser;
    } else {
        gcf(&gammcf, a, x, &gln);
        return gammcf;
    }
}

/* Upper-tail chi-square p-value */
static double chisq_sf(int df, double chi2) {
    return gammq(0.5 * df, 0.5 * chi2);
}

/* ---------------- Poisson pmf via recurrence (stable for small/medium k) ---------------- */
static double poisson_pmf(int k, double lambda) {
    /* compute e^-lambda * lambda^k / k! using recurrence from 0 */
    double p0 = exp(-lambda);
    if (k == 0) return p0;
    double p = p0;
    for (int i = 1; i <= k; i++) {
        p *= lambda / (double)i;
    }
    return p;
}

/* ---------------- Chi-square GOF of observed K's to Poisson(lambda) ----------------
   We follow the Diehard-style rule: combine categories so each expected >= 5.
   Inputs: obs[NO_OBS] = K values (duplicates per sample)
   Outputs: chi2, df, p-value.
*/
static void poisson_chisq_gof(const int *obs, int n, double lambda,
                              double *chi2_out, int *df_out, double *p_out) {
```

```
/* Make a sorted copy of obs */
int *x = (int*)malloc((size_t)n * sizeof(int));
if (!x) { fprintf(stderr, "malloc failed\n"); exit(1); }
memcpy(x, obs, (size_t)n * sizeof(int));

/* sort */
/* simple qsort comparator for int */
int cmp_int(const void *a, const void *b) {
   int A = *(const int*)a, B = *(const int*)b;
   return (A > B) - (A < B);
}
qsort(x, (size_t)n, sizeof(int), cmp_int);

/* Build bins by increasing k, accumulating expected until >=5 */
double chi2 = 0.0;
int bins = 0;

int idx = 0;         /* how many obs consumed */
int k = -1;          /* current Poisson k */
double exp_in_bin = 0.0;
int obs_in_bin = 0;

while (idx < n) {
   /* extend bin until expected >= 5 */
   while (exp_in_bin < 5.0) {
      k++;
      exp_in_bin += (double)n * poisson_pmf(k, lambda);

      /* add all observations equal to k */
      while (idx < n && x[idx] <= k) {
         obs_in_bin++;
```

```
            idx++;
        }

        /* if we are near the end, dump the tail into this bin */
        double remaining_expected = (double)n - (double)(bins == 0 ? 0 : 0); /* not used */
        (void)remaining_expected;

        if (idx >= n) break;
    }

    /* If remaining expected mass is < 5, merge rest into this bin:
       easiest way: if remaining observations are few, keep merging. */
    /* We handle this by checking if remaining expected (tail) would be <5:
       approximate tail expected = n*(1 - sum_{i=0..k} pmf(i)).
       We'll compute tail cdf roughly by summing pmf up to k. */
    /* For simplicity & correctness: if observations remain but we are close,
       we'll continue making bins normally; expected will not stay tiny with lambda=16, n=500. */

    /* finalize this bin */
    if (exp_in_bin > 0.0) {
        double diff = (double)obs_in_bin - exp_in_bin;
        chi2 += (diff * diff) / exp_in_bin;
        bins++;
    }

    /* reset bin counters */
    exp_in_bin = 0.0;
    obs_in_bin = 0;
}

/* degrees of freedom = bins - 1 (lambda fixed) */
```

```
    int df = bins - 1;
    if (df < 1) df = 1;

    double p = chisq_sf(df, chi2);

    free(x);

    *chi2_out = chi2;
    *df_out = df;
    *p_out = p;
}

/* ---------------- One-sample KS test for Uniform(0,1) ---------------- */
static double ks_pvalue_uniform(const double *pvals, int n) {
    double *x = (double*)malloc((size_t)n * sizeof(double));
    if (!x) { fprintf(stderr, "malloc failed\n"); exit(1); }
    memcpy(x, pvals, (size_t)n * sizeof(double));

    int cmp_double(const void *a, const void *b) {
        double A = *(const double*)a, B = *(const double*)b;
        return (A > B) - (A < B);
    }
    qsort(x, (size_t)n, sizeof(double), cmp_double);

    double D = 0.0;
    for (int i = 0; i < n; i++) {
        double xi = x[i];
        double cdf_emp_hi = (double)(i + 1) / (double)n;
        double cdf_emp_lo = (double)(i) / (double)n;
        double d1 = fabs(cdf_emp_hi - xi);
        double d2 = fabs(xi - cdf_emp_lo);
```

```
        if (d1 > D) D = d1;
        if (d2 > D) D = d2;
    }
    free(x);

    /* Kolmogorov asymptotic p-value: p ≈ 2 Σ (-1)^{k-1} exp(-2 k^2 D^2 n) */
    double nn = (double)n;
    double z = D * sqrt(nn);

    /* Use the exact series in terms of D and n */
    double p = 0.0;
    for (int k = 1; k <= 200; k++) {
        double term = exp(-2.0 * (double)k * (double)k * D * D * nn);
        if (k % 2 == 1) p += 2.0 * term;
        else            p -= 2.0 * term;
        if (term < 1e-12) break;
    }
    if (p < 0.0) p = 0.0;
    if (p > 1.0) p = 1.0;
    (void)z;
    return p;
}

/* ---------------- Main Birthday Spacings per bit-window ---------------- */
static void run_bday_window(FILE *fp, int shift, double lambda,
                            double *meanK_out, double *chi2_out, int *df_out, double *p_out) {
    uint32_t bday[NO_BDAY];
    uint32_t space[NO_BDAY];
    int obs[NO_OBS];

    long long sumK = 0;
```

```
for (int t = 0; t < NO_OBS; t++) {
    for (int i = 0; i < NO_BDAY; i++) {
        uint32_t x;
        if (!read_u32_le(fp, &x)) {
            fprintf(stderr, "EOF: not enough data for shift=%d (trial=%d, i=%d)\n", shift, t, i);
            exit(2);
        }
        bday[i] = (x >> shift) & MASK24;
    }

    qsort(bday, NO_BDAY, sizeof(uint32_t), cmp_u32);

    /* Compute spacings INCLUDING wrap-around spacing (critical) */
    for (int i = 0; i < NO_BDAY - 1; i++) {
        space[i] = bday[i + 1] - bday[i];
    }
    space[NO_BDAY - 1] = (bday[0] + N_DAYS) - bday[NO_BDAY - 1];

    qsort(space, NO_BDAY, sizeof(uint32_t), cmp_u32);

    /* Count duplicates (total repeats) */
    int K = 0;
    for (int i = 1; i < NO_BDAY; i++) {
        if (space[i] == space[i - 1]) K++;
    }

    obs[t] = K;
    sumK += K;
}
```

```
    *meanK_out = (double)sumK / (double)NO_OBS;

    /* Chi-square goodness-of-fit of obs[] to Poisson(lambda) */
    poisson_chisq_gof(obs, NO_OBS, lambda, chi2_out, df_out, p_out);
}

int main(int argc, char **argv) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s output.dat\n", argv[0]);
        return 1;
    }

    const char *fname = argv[1];
    FILE *fp = fopen(fname, "rb");
    if (!fp) {
        perror("fopen");
        return 1;
    }

    /* lambda = m^3/(4n) with m=1024, n=2^24 => 16 */
    double lambda = (double)NO_BDAY * (double)NO_BDAY * (double)NO_BDAY / (4.0 *
(double)N_DAYS);

    printf("\nDiehard Birthday Spacings Test on file: %s\n", fname);
    printf("n = 2^%d = %u, m = %d, samples = %d, lambda = %.6f\n\n",
           NO_BITS, N_DAYS, NO_BDAY, NO_OBS, lambda);

    printf("Bits used (from MSB view)\tmean(K)\t\tchisq\t\tdf\tp-value\n");

    double pvals[9];
    int idx = 0;
```

```
/* Diehard uses bits 1-24, 2-25, ... 9-32.
   With uint32 x, that corresponds to shifts 8 down to 0 if you imagine MSB-indexing.
   But mathematically it is just 9 consecutive 24-bit windows.
   We implement shift = 0..8 (LSB upward). This is standard in many implementations.
   If you want "from the left" exactly, reverse the shift order—KS result is the same set.
*/

for (int shift = 0; shift <= 8; shift++) {
    rewind(fp);

    double meanK, chi2, p;
    int df;
    run_bday_window(fp, shift, lambda, &meanK, &chi2, &df, &p);

    /* report bits (1-based human display) */
    int bit_lo = shift + 1;
    int bit_hi = shift + NO_BITS;

    printf("%2d to %2d\t\t\t\t%.3f\t\t%.4f\t\t%d\t%.6g\n",
          bit_lo, bit_hi, meanK, chi2, df, p);

    pvals[idx++] = p;
}

fclose(fp);

/* KS test on the 9 p-values */
double ks_p = ks_pvalue_uniform(pvals, 9);
printf("\nKS test on the 9 p-values: KS p-value = %.6g\n\n", ks_p);
```

```
    return 0;
}



  /*
  Clean Diehard BIRTHDAY SPACINGS test

  Parameters (classic Diehard):
    n = 2^24 days
    m = 1024 birthdays per sample
    no_obs = 500 samples per bit-window
    windows = 9 (24-bit windows across 32-bit words)

  For each window:
    - compute K (duplicate spacings) for each sample
    - do chi-square GOF to Poisson(lambda = m^3/(4n) = 16)
    - output p-value
  Then:
    - KS test on the 9 p-values

  Build: gcc -O2 -std=c11 bday_clean.c -lm
  Run:   ./a.out output.dat
*/

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <math.h>
```

```
#define NO_OBS   500
#define NO_BDAY  1024
#define NO_BITS  24
#define N_DAYS   (1u << 24)
#define MASK24   0xFFFFFFu
#define WINDOWS  (32 - NO_BITS + 1)  /* 9 */

static int cmp_u32(const void *a, const void *b) {
   uint32_t x = *(const uint32_t*)a, y = *(const uint32_t*)b;
   return (x > y) - (x < y);
}
static int cmp_d(const void *a, const void *b) {
   double x = *(const double*)a, y = *(const double*)b;
   return (x > y) - (x < y);
}
static int cmp_i(const void *a, const void *b) {
   int x = *(const int*)a, y = *(const int*)b;
   return (x > y) - (x < y);
}

/* -------- read one little-endian uint32 from raw byte file -------- */
static int read_u32_le(FILE *fp, uint32_t *out) {
   unsigned char b[4];
   if (fread(b, 1, 4, fp) != 4) return 0;
   *out = (uint32_t)b[0]
        | ((uint32_t)b[1] << 8)
        | ((uint32_t)b[2] << 16)
        | ((uint32_t)b[3] << 24);
   return 1;
}
```

```
/* -------- Regularized upper incomplete gamma Q(a,x) for chi-square SF -------- */
/* (Numerical Recipes style implementation; stable and standard.) */
static double gammln(double xx) {
   static const double cof[6] = {
      76.18009172947146,    -86.50532032941677,
      24.01409824083091,    -1.231739572450155,
      0.1208650973866179e-2,-0.5395239384953e-5
   };
   double x = xx, y = xx;
   double tmp = x + 5.5;
   tmp -= (x + 0.5) * log(tmp);
   double ser = 1.000000000190015;
   for (int j = 0; j < 6; j++) ser += cof[j] / ++y;
   return -tmp + log(2.5066282746310005 * ser / x);
}
static void gser(double *gamser, double a, double x, double *gln) {
   const int ITMAX = 200;
   const double EPS = 3.0e-14;
   *gln = gammln(a);
   if (x <= 0.0) { *gamser = 0.0; return; }
   double sum = 1.0 / a, del = sum, ap = a;
   for (int n = 1; n <= ITMAX; n++) {
      ap += 1.0;
      del *= x / ap;
      sum += del;
      if (fabs(del) < fabs(sum) * EPS) break;
   }
   *gamser = sum * exp(-x + a * log(x) - (*gln));
}
static void gcf(double *gammcf, double a, double x, double *gln) {
   const int ITMAX = 200;
```

```
    const double EPS = 3.0e-14;
    const double FPMIN = 1.0e-300;
    *gln = gammln(a);
    double b = x + 1.0 - a;
    double c = 1.0 / FPMIN;
    double d = 1.0 / b;
    double h = d;
    for (int i = 1; i <= ITMAX; i++) {
        double an = -i * (i - a);
        b += 2.0;
        d = an * d + b;  if (fabs(d) < FPMIN) d = FPMIN;
        c = b + an / c;  if (fabs(c) < FPMIN) c = FPMIN;
        d = 1.0 / d;
        double del = d * c;
        h *= del;
        if (fabs(del - 1.0) < EPS) break;
    }
    *gammcf = exp(-x + a * log(x) - (*gln)) * h;
}
static double gammq(double a, double x) {
    if (x < 0.0 || a <= 0.0) return 0.0;
    double gamser, gammcf, gln;
    if (x < a + 1.0) { gser(&gamser, a, x, &gln); return 1.0 - gamser; }
    else             { gcf(&gammcf, a, x, &gln); return gammcf; }
}
static double chisq_sf(int df, double chi2) {
    return gammq(0.5 * df, 0.5 * chi2);
}

/* -------- Poisson pmf by recurrence: p(k) = e^-λ λ^k / k! -------- */
static double poisson_pmf(int k, double lambda) {
```

```
    double p = exp(-lambda);
    for (int i = 1; i <= k; i++) p *= lambda / (double)i;
    return p;
}

/* -------- Chi-square GOF to Poisson(lambda), bins combined so expected >= 5 --------
   Returns p-value, and outputs chi2 and df.
*/
static double poisson_chisq_gof(const int *obs, int n, double lambda, double *chi2_out, int *df_out)
{
    int *x = (int*)malloc((size_t)n * sizeof(int));
    if (!x) { fprintf(stderr, "malloc failed\n"); exit(1); }
    memcpy(x, obs, (size_t)n * sizeof(int));
    qsort(x, (size_t)n, sizeof(int), cmp_i);

    double chi2 = 0.0;
    int bins = 0;

    int idx = 0;
    int k = -1;

    while (idx < n) {
        double E = 0.0;
        int O = 0;

        /* grow a bin until E >= 5 */
        while (E < 5.0) {
            k++;
            E += (double)n * poisson_pmf(k, lambda);
            while (idx < n && x[idx] <= k) { O++; idx++; }
```

```
            /* if we already consumed all obs, stop */
            if (idx >= n) break;
        }

        if (E > 0.0) {
            double d = (double)O - E;
            chi2 += (d * d) / E;
            bins++;
        }
    }

    free(x);

    int df = bins - 1;          /* lambda fixed (not estimated) */
    if (df < 1) df = 1;

    *chi2_out = chi2;
    *df_out = df;
    return chisq_sf(df, chi2);
}

/* -------- 1-sample KS test against Uniform(0,1), asymptotic p-value -------- */
static double ks_uniform_pvalue(const double *pvals, int n) {
    double *x = (double*)malloc((size_t)n * sizeof(double));
    if (!x) { fprintf(stderr, "malloc failed\n"); exit(1); }
    memcpy(x, pvals, (size_t)n * sizeof(double));
    qsort(x, (size_t)n, sizeof(double), cmp_d);

    double D = 0.0;
    for (int i = 0; i < n; i++) {
        double xi = x[i];
```

```
        double F_hi = (double)(i + 1) / (double)n;
        double F_lo = (double)(i) / (double)n;
        double d1 = fabs(F_hi - xi);
        double d2 = fabs(xi - F_lo);
        if (d1 > D) D = d1;
        if (d2 > D) D = d2;
    }
    free(x);

    /* Kolmogorov asymptotic series */
    double nn = (double)n;
    double p = 0.0;
    for (int k = 1; k <= 200; k++) {
        double term = exp(-2.0 * (double)k * (double)k * D * D * nn);
        p += (k % 2 ? 2.0 : -2.0) * term;
        if (term < 1e-12) break;
    }
    if (p < 0.0) p = 0.0;
    if (p > 1.0) p = 1.0;
    return p;
}

/* -------- compute one window: returns p-value for GOF, plus meanK, chi2, df -------- */
static void birthday_test_window(FILE *fp, int shift,
                                 double lambda,
                                 double *meanK_out, double *chi2_out, int *df_out, double *p_out)
{
    uint32_t bday[NO_BDAY];
    uint32_t sp[NO_BDAY];
    int obs[NO_OBS];
    long long sumK = 0;
```

```
    for (int t = 0; t < NO_OBS; t++) {
        for (int i = 0; i < NO_BDAY; i++) {
            uint32_t x;
            if (!read_u32_le(fp, &x)) {
                fprintf(stderr, "EOF: not enough data (shift=%d, sample=%d, i=%d)\n", shift, t, i);
                exit(2);
            }
            bday[i] = (x >> shift) & MASK24;
        }

        qsort(bday, NO_BDAY, sizeof(uint32_t), cmp_u32);

        /* spacings INCLUDING wrap-around spacing (critical) */
        for (int i = 0; i < NO_BDAY - 1; i++) sp[i] = bday[i + 1] - bday[i];
        sp[NO_BDAY - 1] = (bday[0] + N_DAYS) - bday[NO_BDAY - 1];

        qsort(sp, NO_BDAY, sizeof(uint32_t), cmp_u32);

        int K = 0;
        for (int i = 1; i < NO_BDAY; i++) if (sp[i] == sp[i - 1]) K++;

        obs[t] = K;
        sumK += K;
    }

    *meanK_out = (double)sumK / (double)NO_OBS;
    *p_out = poisson_chisq_gof(obs, NO_OBS, lambda, chi2_out, df_out);
}

int main(int argc, char **argv) {
```

```
   if (argc < 2) {
      fprintf(stderr, "Usage: %s output.dat\n", argv[0]);
      return 1;
   }

   const char *filename = argv[1];
   FILE *fp = fopen(filename, "rb");
   if (!fp) { perror("fopen"); return 1; }

   const double lambda = (double)NO_BDAY * (double)NO_BDAY * (double)NO_BDAY / (4.0 *
(double)N_DAYS);

   puts("\n\t|---------------------------------------------------------------|");
   puts("\t|          BIRTHDAY SPACINGS TEST (Clean C Version)         |");
   puts("\t|n=2^24, m=1024. For each 24-bit window, compute duplicates K |");
   puts("\t|in spacing list and test K ~ Poisson(lambda=m^3/(4n)=16).     |");
   puts("\t|Nine windows => 9 p-values => KS test for Uniform(0,1).       |");
   puts("\t|---------------------------------------------------------------|\n");

   printf("\tRESULTS FOR %s\n", filename);
   printf("\t(m=%d, n=2^%d, lambda=%.2f, samples=%d, windows=%d)\n\n",
        NO_BDAY, NO_BITS, lambda, NO_OBS, WINDOWS);

   printf("\tBits used\tmean(K)\t\tchisq\t\tdf\tp-value\n");

   double pvals[WINDOWS];

   for (int shift = 0; shift < WINDOWS; shift++) {
      /* Diehard reuses same file for each window: close/reopen or rewind */
      rewind(fp);
```

```
        double meanK, chi2, p;
        int df;

        birthday_test_window(fp, shift, lambda, &meanK, &chi2, &df, &p);

        int bit_lo = shift + 1;
        int bit_hi = shift + NO_BITS;

        printf("\t%2d to %2d\t%.2f\t\t%.4f\t\t%d\t%.6g\n",
              bit_lo, bit_hi, meanK, chi2, df, p);

        pvals[shift] = p;
    }

    fclose(fp);

    double ks_p = ks_uniform_pvalue(pvals, WINDOWS);
    printf("\n\tKS p-value on %d p-values = %.6g\n\n", WINDOWS, ks_p);

    return 0;
}
```